Verification and Validation Report: AutoVox

Team #10, Five of a Kind
Omar Abdelhamid
Daniel Maurer
Andrew Bovbel
Olivia Reich
Khalid Farag

March 8, 2026

1 Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

2 Symbols, Abbreviations and Acronyms

symbol	description
T	Test
TC	Test Case
FR	Functional Requirement
NF	Non-Functional Requirement
VnV	Verification and Validation
IM	Import Manager
VM	Visualization Manager
EM	Editing Manager
XM	Export Manager
MAX_VOXELS	Maximum number of voxels to display in the 3D view
STL	Standard Tessellation Language

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**

4 Nonfunctional Requirements Evaluation **1**
4.1 Usability . . . 1
4.2 Performance . . . 1
4.3 etc. . . . 1

5 Comparison to Existing Implementation **1**

6 Unit Testing **1**
6.1 Import Manager Tests . . . 1
6.1.1 CAD File Import and Project Management . . . 1
6.1.2 Model Configuration and Slicing . . . 2
6.2 Visualization Manager Tests . . . 4
6.2.1 3D Model Rendering and Display . . . 4
6.2.2 Layer Focus and Voxel Selection Tests . . . 5
6.2.3 Material and Magnetization Tracking . . . 6
6.3 Editing Manager Tests . . . 7
6.3.1 Editing a voxel . . . 7
6.3.2 Utilizing the history manager . . . 11
6.3.3 Saving project . . . 13
6.4 Export Manager Tests . . . 14
6.4.1 Export Validation and File Export . . . 14

7 Changes Due to Testing **16**
7.1 Usability and User Input Adjustments . . . 16
7.2 Interaction Flow Adjustments . . . 17
7.3 Performance Adjustments . . . 17
7.4 Future Improvements . . . 17

8 Automated Testing **18**

9 Trace to Requirements **18**
9.1 Functional Requirements . . . 18
9.2 Non-Functional Requirements . . . 22

10 Trace to Modules **23**

11 Code Coverage Metrics **27**
11.1 Python Code Coverage . . . 27
11.2 TypeScript Code Coverage . . . 27

List of Tables

1 Import Manager: STL File Import and Project Management Test Cases . . . 2
2 Import Manager: Model Configuration and Slicing Test Cases 3
3 Visualization Manager: 3D Model Rendering and Display Test Cases 4
4 Visualization Manager: Layer Focus and Voxel Selection Test Cases 5
5 Visualization Manager: Material and Magnetization Tracking Test Cases . . 6
6 Editing Manager: Editing a voxel (system and backend) 7
7 Editing Manager: Utilizing the history manager (backend) 12
8 Editing Manager: Saving project (backend) 14
9 Export Manager Test Cases 15
10 Trace Between Import Manager Test Cases and Corresponding Requirements 19
11 Trace Between Visualization Manager test cases and corresponding requirements 19
12 Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 1: voxel editing and history) 20
13 Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 2: layers, add/delete, reset, save) 21
14 Trace Between Export Manager Test Cases and Corresponding Requirements 21
15 Trace Between Non-Functional Requirements Import Manager Test Cases and Corresponding Requirements 22
16 Trace Between Non-Functional Requirements Visualization Manager Test Cases and Corresponding Requirements 22
17 Trace Between Non-Functional Requirements Editing Manager Test Cases and Corresponding Requirements 22
18 Trace Between Non-Functional Requirements Export Manager Test Cases and Corresponding Requirements 23
19 Trace Between Import Manager Test Cases and Modules 23
20 Trace Between Visualization Manager Test Cases and Modules 24
21 Trace Between Editing Manager Test Cases and Modules (Part 1: voxel editing and history) 25
22 Trace Between Editing Manager Test Cases and Modules (Part 2: layers, add/delete, reset, save) 26
23 Trace Between Export Manager Test Cases and Modules 26

List of Figures

1 Python Code Coverage 27
2 TypeScript Code Coverage 28

This document ...

3 Functional Requirements Evaluation

4 Nonfunctional Requirements Evaluation

4.1 Usability

4.2 Performance

4.3 etc.

5 Comparison to Existing Implementation

This section will not be appropriate for every project.

6 Unit Testing

This section reports **test results** and evidence for the system unit tests. The emphasis is on outcomes and evidence, not on the rationale for test selection (see the VnV Plan for that). Test cases are organized by manager: Import Manager (TC1–TC5), Visualization Manager (TC4–TC11), Editing Manager (TC12–TC61), and Export Manager (TC62–TC69).

For each test case, the tables below provide: **test case name** (TC id), **initial state** (implied by the Ref. Req. and test context), **input** (Action), **expected result**, and **whether actual output matched expected** (Result: Pass/Fail). Actual outcomes are backed by automated test runs; see Section 8 for how to reproduce.

6.1 Import Manager Tests

6.1.1 CAD File Import and Project Management

Table 1 gives the results for STL file import and project management tests (TC1–TC5).

Table 1: Import Manager: STL File Import and Project Management Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC1	F211	Provide a valid STL file to system through file settings panel	STL file is uploaded succesfully to the system, and saved to the sample file directory	All assertions are met and passed by backend unit test	Pass
TC2	F211	Browse all uploaded STL files in the sample file directory	System displays a list of all uploaded STL files in the sample file directory	All assertions are met and passed by backend unit test	Pass
TC3	F211	Browse all uploaded STL files, without ever uploading a STL file	System displays an empty list of uploaded STL files	All assertions are met and passed by backend unit test	Pass
TC4	F211	Provide a file with unsupported format (e.g. .txt, .jpg, .pdf) to system through file settings panel	System rejects the file and displays an error indicating unsupported format	All assertions are met and passed by backend unit test	Pass
TC5	F212	Open a previously uploaded STL file	System loads the STL file from the sample file directory	All assertions are met and passed by backend unit test	Pass

6.1.2 Model Configuration and Slicing

Table 2 gives the results for model configuration and slicing tests (TC6–TC12).

Table 2: Import Manager: Model Configuration and Slicing Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
//Missing TC6	F213	Specify custom voxel size	System applies the dimensions and slices the model into voxels of the requested size	All assertions are met and passed by backend unit test	Pass
//Missing TC7	F214	Specify custom geometric model size	System scales the model to desired model size before slicing	All assertions are met and passed by backend unit test	Pass
TC8	F216	Import a model containing more than MAX_VOXELS voxels.	System partitions the model into display sections not exceeding MAX_VOXELS	All assertions are met and passed by backend unit test	Pass
TC9	F216	Import a model containing less than MAX_VOXELS voxels	System returns only one partition with all voxels visible	All assertions are met and passed by backend unit test	Pass
TC10	F216	Delete a partition with no voxels	System deletes the partition and returns the model to the previous state	All assertions are met and passed by backend unit test	Pass
TC11	F213, F214, F215	Voxelize uploaded STL model into voxels	System voxelizes the model into voxels of the requested size using Triangle Meshes	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC12	F213, F214, F215	Voxelize non-existing STL model into voxels	System returns an error indicating the STL model does not exist	All assertions are met and passed by backend unit test	Pass

6.2 Visualization Manager Tests

6.2.1 3D Model Rendering and Display

Table 3 gives the results for 3D model rendering and display tests (TC13–TC16).

Table 3: Visualization Manager: 3D Model Rendering and Display Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC13	F221	Import a STL model into the system	System renders a non-voxelized 3D STL model using Three.js	All assertions are met and passed by frontend unit test	Pass
TC14	F221	Open a voxelized project from file directory	System renders a voxelized 3D model using Three.js broken down into layers	All assertions are met and passed by frontend unit test	Pass
TC15	F222	Open a voxelized project from file directory where the number of voxles is greater than MAX_VOXELS	System displays a partitions panel with more than 1 partition	All assertions are met and passed by frontend unit test	Pass
TC16	F222	Open a voxelized project from file directory where the number of voxles is less than MAX_VOXELS	System displays a partitions panel with only 1 partition	All assertions are met and passed by frontend unit test	Pass

6.2.2 Layer Focus and Voxel Selection Tests

Table 4 gives the results for layer focus and voxel selection tests (TC17–TC24).

Table 4: Visualization Manager: Layer Focus and Voxel Selection Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC17	F222	Pick a specific partition to focus on from the partitions panel.	The selected partition is isolated and displayed as a 3D view without any other partitions.	All assertions are met and passed by frontend unit test	Pass
TC18	F223	Rotate the model using interface controls.	Model rotates smoothly and intuitively	All assertions are met and passed by frontend unit test	Pass
TC19	F223	Zoom the model using interface controls.	Model zooms smoothly and intuitively	All assertions are met and passed by frontend unit test	Pass
TC20	F224	Select a specific layer to focus on from the 3D model display.	Selected layer is isolated, layer editor is opened displaying the layer voxels and properties in a 2D view.	All assertions are met and passed by frontend unit test	Pass
TC21	F225	Select individual voxel within the focused layer from the layer editor.	The selected voxel is highlighted in the 2D layer view	All assertions are met and passed by frontend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC22	F225	Select multiple voxels within the focused layer from the layer editor using lasso selection.	The selected voxels are highlighted in the 2D layer view	All assertions are met and passed by frontend unit test	Pass
TC23	F225	Select all voxels in the partition on the 3D view (CTRL+A)	All voxels in the partition are highlighted in the 3D view	All assertions are met and passed by frontend unit test	Pass
TC24	F225	Select all voxels in the layer on the 2D layer view (CTRL+A)	All voxels in the layer are highlighted in the 2D layer view	All assertions are met and passed by frontend unit test	Pass

6.2.3 Material and Magnetization Tracking

Table 5 gives the results for material and magnetization tracking tests (TC25–TC26).

Table 5: Visualization Manager: Material and Magnetization Tracking Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC25	F226	Assign material IDs to a voxel or multiple voxels in the layer editor.	The color of the voxels are updated to the assigned material color.	All assertions are met and passed by frontend unit test	Pass
TC26	F227	Assign magnetization to a voxel or multiple voxels in the layer editor.	The voxels assigned will be highlighted with a X-arrow icon indicating magnetization.	All assertions are met and passed by frontend unit test	Pass

6.3 Editing Manager Tests

The Editing Manager tests are organized into three subsections: editing a voxel (TC27–TC49), utilizing the history manager (TC50–TC58), and saving project (TC59–TC62).

6.3.1 Editing a voxel

Table 6 gives the results for voxel editing tests (TC27–TC49): magnetization and material assignment, layer access, and adding, deleting, or resetting voxels.

Table 6: Editing Manager: Editing a voxel (system and backend)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC27	F231	Update magnetization of a single voxel (e.g. x=1.0, y=0.0, z=0.0).	System saves the magnetization vector of the voxel to the database	All assertions are met and passed by backend unit test	Pass
TC28	F231	Update magnetization of a multiple voxels (e.g. x=1.0, y=0.0, z=0.0).	System saves the magnetization vector of the voxels to the database	All assertions are met and passed by backend unit test	Pass
TC29	F231	Update magnetization of a non-existing voxel (e.g. x=1.0, y=0.0, z=0.0).	System returns an error indicating the voxel does not exist	All assertions are met and passed by backend unit test	Pass
TC30	F231	Update magnetization of a voxel with invalid magnetization vector (e.g. x=1.0, y=0.0, z=0.0).	System returns an error indicating the magnetization vector is invalid	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC31	F232	Create a favourite magnetization vector (e.g. x=1.0, y=0.0, z=0.0).	System saves the magnetization vector to the favourite bar	All assertions are met and passed by backend unit test	Pass
TC32	F233	Assign material ID to a single voxel (e.g. 2).	System saves the material ID to the voxel and in the database	All assertions are met and passed by backend unit test	Pass
TC33	F233	Assign material ID to a multiple voxels (e.g. 2).	System saves the material ID to the voxels and in the database	All assertions are met and passed by backend unit test	Pass
TC34	F233	Assign material ID to a non-existing voxel (e.g. 2).	System returns an error indicating the voxel does not exist	All assertions are met and passed by backend unit test	Pass
TC35	F231, F233	Assign material ID to a voxel and magnetization to a voxel (e.g. 2, x=1.0, y=0.0, z=0.0).	System saves the material ID and magnetization vector to the voxel and in the database	All assertions are met and passed by backend unit test	Pass
TC36	F234	Assign label "Steel" to material ID 1.	Material ID 1 is associated with the label "Steel" in the database.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC37	F235	Copy assigned properties from one layer to another layer.	Target layer voxels receive the same material and magnetization properties as the source.	All assertions are met and passed by backend and frontend unit test	Pass
TC38	F238	Get list of layers in the project.	The list of layers is returned.	All assertions are met and passed by backend unit test	Pass
TC39	F238	Get specific layer in the project based on the layer index and axis.	The specific layer is returned.	All assertions are met and passed by backend unit test	Pass
TC40	F238	Get specific layer with non-existing layer index.	System returns an error indicating the layer index does not exist.	All assertions are met and passed by backend unit test	Pass
TC41	F238	Get specific layer with non-existing axis.	System returns an error indicating the axis does not exist.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC42	F238	Get specific layer with non-existing project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC43	F238	Get specific layer with non-existing partition.	System returns an error indicating the partition does not exist.	All assertions are met and passed by backend unit test	Pass
TC44	F239	Add a new voxel at new indices.	A new voxel is added to the database at the new indices with default material and magnetization.	All assertions are met and passed by backend and frontend unit test	Pass
TC45	F239	Delete an existing voxel.	A voxel is removed from the database at the existing indices.	All assertions are met and passed by backend and frontend unit test	Pass
TC46	F239	Delete a non-existent voxel.	System returns an error indicating the voxel does not exist.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC47	F2310	Reset materials on selected voxels.	The voxels are reset to the default material in the database.	All assertions are met and passed by backend and frontend unit test	Pass
TC48	F2310	Reset magnetizations on selected voxels.	The voxels are reset to the default magnetization in the database.	All assertions are met and passed by backend and frontend unit test	Pass
TC49	F2310	Reset materials and magnetizations on selected voxels.	The voxels are reset to the default material and magnetization in the database.	All assertions are met and passed by backend and frontend unit test	Pass

6.3.2 Utilizing the history manager

Table 7 gives the results for history manager tests (TC50–TC58): undo, redo, and clear history.

Table 7: Editing Manager: Utilizing the history manager (backend)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC50	F237	Undo any updates made to a voxel.	The changes are undone and the voxel returns to the previous state.	All assertions are met and passed by backend unit test	Pass
TC51	F237	Redo any undone updates to a voxel.	The changes are redone and the voxel returns to the previous state.	All assertions are met and passed by backend unit test	Pass
TC52	F237	Undo any added voxels.	The voxels are removed from the database.	All assertions are met and passed by backend unit test	Pass
TC53	F237	Redo any deleted voxels.	The voxels are added back to the database.	All assertions are met and passed by backend unit test	Pass
TC54	F237	Undo any reset material on a voxel.	The voxel is reset to the default material.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC55	F237	Redo any reset material on a voxel.	The voxel is reset to the default material.	All assertions are met and passed by backend unit test	Pass
TC56	F237	Undo when history empty.	System returns an error indicating the history is empty	All assertions are met and passed by backend unit test	Pass
TC57	F237	Redo when redo stack empty.	System returns an error indicating the redo stack is empty	All assertions are met and passed by backend unit test	Pass
TC58	F237	Clear history.	Both history and redo stacks empty.	All assertions are met and passed by backend unit test	Pass

6.3.3 Saving project

Table 8 gives the results for saving project tests (TC59–TC62).

Table 8: Editing Manager: Saving project (backend)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC59	F236	Save the project automatically.	The project is saved to the database.	All assertions are met and passed by backend unit test	Pass
TC60	F2311	Save the project manually.	The project is saved to the database.	All assertions are met and passed by backend unit test	Pass
TC61	F2311	Save a non-existent project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC62	F2311	Save a project with invalid data.	System returns an error indicating the project data is invalid.	All assertions are met and passed by backend unit test	Pass

6.4 Export Manager Tests

6.4.1 Export Validation and File Export

Table 9 gives the results for export validation and file export (TC63–TC72).

Table 9: Export Manager Test Cases

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC63	F241	Validate all voxels have magnetization.	System checks if all voxels have magnetization and returns an error if any voxels are missing magnetization.	All assertions are met and passed by backend unit test	Pass
TC64	F241	Validate all voxels have material.	System checks if all voxels have material and returns an error if any voxels are missing material.	All assertions are met and passed by backend unit test	Pass
TC65	F242	Export CSV file of an existing project.	System exports a CSV file with all voxel metadata.	All assertions are met and passed by backend unit test	Pass
TC66	F242	Export CSV file of a non-existent project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC67	F242	Export CSV file of a project with invalid data.	System returns an error indicating the project data is invalid.	All assertions are met and passed by backend unit test	Pass
TC68	F243	Export project file of an existing project.	System exports a project file with all project data.	All assertions are met and passed by backend unit test	Pass

Continued on next page

(Continued from previous page)

TC	Ref. Req.	Action	Expected Result	Actual Result	Result
TC69	F243	Export project file of a non-existent project.	System returns an error indicating the project does not exist.	All assertions are met and passed by backend unit test	Pass
TC70	F243	Export project file of a project with invalid data.	System returns an error indicating the project data is invalid.	All assertions are met and passed by backend unit test	Pass
TC71	F245	Export model with summary option enabled.	System exports a model file with a summary of the model.	All assertions are met and passed by backend unit test	Pass
TC72	F241	Export operation is interrupted (simulated failure).	Original project data remains intact and unaltered.	All assertions are met and passed by backend unit test	Pass

7 Changes Due to Testing

During the VnV process, several changes to the project that were made based on the feedback from the stakeholders (supervisor and graduate students) that will be used to improve the system. These changes are related to the usability and user input, interaction flow, and performance adjustments.

7.1 Usability and User Input Adjustments

Feedback from usability testing led to the following usability and user-input changes:

- **Project and file menus:** A project popup after selecting a file to show existing projects, improving feedback and selection, as it was not clear where to naviagte to open a project.
- **Layer editor:** The magnitude control in the layer editor was removed to reduce clutter and confusion and as it was not used by the stakeholders.

- **File and material naming:** File naming was reviewed to handle spaces and to indicate when a file with the same name already exists. Material naming in settings was reviewed for clarity and consistency.
- **Magnetization display:** A checkmark or toggle was added to show or hide magnetization, giving users control over what is displayed.
- **Partition behaviour:** Default partition centring was changed so partitions are not automatically centred. A rename-partition capability was added.

7.2 Interaction Flow Adjustments

The following changes address the sequence of user actions and overall flow from opening a file to working with partitions:

- **Partition visibility:** The cause of partitions not showing immediately was investigated and addressed so that partitions appear promptly after selecting a project or confirguring a new one.
- **Reduced redundancy:** When a file is already open, re-selecting or re-confirming the same file is no longer required for subsequent actions where possible.

7.3 Performance Adjustments

Performance feedback from testing led to optimisations focused on model loading and rendering:

- **Flow from first open to partitions:** The slowness of the flow from first opening a file to obtaining partitions was addressed through performance improvements in file loading and initial processing.
- **Model rendering:** Trimesh (or equivalent mesh handling) was used or integrated to speed up model rendering, reducing latency when displaying the voxelised model and partitions.

7.4 Future Improvements

Usability testing also highlighted potential enhancements for future work:

- **Partition logic:** Revisit or extend partition logic to support more flexible partitioning strategies and user control.
- **Lasso tool:** Consider a lasso (or similar) selection tool to add voxels in a more intuitive way.
- **Export Model Summary:** Add a model summary to the export model file to allow users to see what changes were made to the model.

- **Export Progress Trackers:** Test the export performance of the system to ensure it meets the requirements and improve the progress tracker.

Overall, the testing phase was successfull and played a crucial role in the development of the system. It allowed for the identification of issues and the implementation of improvements, ensuring the system is of high quality and meets the requirements of the stakeholders.

8 Automated Testing

All tests for the Python backend and for the TypeScript frontend modules are automated. The pass/fail results in the tables in Section 6 are backed by the output of these test runs, readers can reproduce the evidence by running the tests as described in the VnV Plan.

- **Backend (Python/PyTest):** From the project root, run `pytest` in `src/backend/` (e.g. `cd src/backend && pytest`). The console output shows per-test pass/fail and any failures; this output is the evidence for the "Actual Result" and "Result" columns in the tables in Section 6.
- **Frontend (TypeScript/Jest):** From `src/frontend/`, run `npm run test`. The console (or test report) output is the evidence for the "Actual Result" and "Result" columns in the tables in Section 6.

System-level and UI tests are executed manually; their outcomes are recorded in the corresponding tables in Section 6 and match the expected results listed there.

9 Trace to Requirements

The test cases in this report are the execution of the system and unit tests defined in the VnV Plan. The following tables show the traceability between these test cases and the functional requirements from the SRS. A filled cell (■) indicates that the test case verifies the corresponding requirement; an empty cell (□) indicates it does not.

9.1 Functional Requirements

Table 10 shows the trace between Import Manager test cases and corresponding requirements.

Table 10: Trace Between Import Manager Test Cases and Corresponding Requirements

Reqs:	F211	F212	F213	F214	F215	F216
TC1	■	□	□	□	□	□
TC2	■	□	□	□	□	□
TC3	■	□	□	□	□	□
TC4	■	□	□	□	□	□
TC5	□	■	□	□	□	□
TC6	□	□	■	□	□	□
TC7	□	□	□	■	□	□
TC8	□	□	□	□	□	■
TC9	□	□	□	□	□	■
TC10	□	□	□	□	□	■
TC11	□	□	■	■	■	□
TC12	□	□	■	■	■	□

Table 11 shows the trace between Visualization Manager test cases and corresponding requirements.

Table 11: Trace Between Visualization Manager test cases and corresponding requirements

Reqs:	F221	F222	F223	F224	F225	F226	F227
TC13	■	□	□	□	□	□	□
TC14	■	□	□	□	□	□	□
TC15	□	■	□	□	□	□	□
TC16	□	■	□	□	□	□	□
TC17	□	■	□	□	□	□	□
TC18	□	□	■	□	□	□	□
TC19	□	□	■	□	□	□	□
TC20	□	□	□	■	□	□	□
TC21	□	□	□	□	■	□	□
TC22	□	□	□	□	■	□	□
TC23	□	□	□	□	■	□	□
TC24	□	□	□	□	■	□	□
TC25	□	□	□	□	□	■	□
TC26	□	□	□	□	□	□	■

Table 12 shows the trace between Editing Manager test cases and corresponding require-

ments (Part 1: voxel editing and history).

Table 12: Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 1: voxel editing and history)

Reqs:	F231	F232	F233	F234	F235	F237
TC27	■	□	□	□	□	□
TC28	■	□	□	□	□	□
TC29	■	□	□	□	□	□
TC30	■	□	□	□	□	□
TC31	□	■	□	□	□	□
TC32	□	□	■	□	□	□
TC33	□	□	■	□	□	□
TC34	□	□	■	□	□	□
TC35	■	□	■	□	□	□
TC36	□	□	□	■	□	□
TC37	□	□	□	□	■	□
TC38	□	□	□	□	□	■
TC39	□	□	□	□	□	■
TC40	□	□	□	□	□	■
TC41	□	□	□	□	□	■
TC42	□	□	□	□	□	■
TC43	□	□	□	□	□	■
TC44	□	□	□	□	□	■
TC45	□	□	□	□	□	■
TC46	□	□	□	□	□	■

Table 13 shows the trace between Editing Manager test cases and corresponding requirements (Part 2: layers, add/delete, reset, save).

Table 13: Trace Between Editing Manager Test Cases and Corresponding Requirements (Part 2: layers, add/delete, reset, save)

Reqs:	F238	F239	F2310	F2311
TC47	■	□	□	□
TC48	■	□	□	□
TC49	■	□	□	□
TC50	■	□	□	□
TC51	■	□	□	□
TC52	■	□	□	□
TC53	□	■	□	□
TC54	□	■	□	□
TC55	□	■	□	□
TC56	□	□	■	□
TC57	□	□	■	□
TC58	□	□	■	□
TC59	□	□	□	■
TC60	□	□	□	■
TC61	□	□	□	■
TC62	□	□	□	■

Table 14 shows the trace between Export Manager test cases and corresponding requirements.

Table 14: Trace Between Export Manager Test Cases and Corresponding Requirements

Reqs:	F241	F242	F243	F244	F245
TC63	■	□	□	□	□
TC64	■	□	□	□	□
TC65	□	■	□	□	□
TC66	□	■	□	□	□
TC67	□	■	□	□	□
TC68	□	□	■	□	□
TC69	□	□	■	□	□
TC70	□	□	■	□	□
TC71	□	□	□	□	■
TC72	■	□	□	□	□

9.2 Non-Functional Requirements

Table 15 shows the trace between Non-Functional Requirements Import Manager test cases and corresponding requirements.

Table 15: Trace Between Non-Functional Requirements Import Manager Test Cases and Corresponding Requirements

Reqs:	NF211	NF212	NF213
TC8	■	□	□
TC9	■	□	□
Not Explicitly Tested	□	■	□
Tested by usability test (Graduate Students)	□	□	■

Table 16 shows the trace between Non-Functional Requirements Visualization Manager test cases and corresponding requirements.

Table 16: Trace Between Non-Functional Requirements Visualization Manager Test Cases and Corresponding Requirements

Reqs:	NF221	NF222	NF223
TC18	■	□	□
TC19	■	□	□
TC15	□	■	□
TC16	□	■	□
Not Explicitly Tested	□	□	■

Table 17 shows the trace between Non-Functional Requirements Editing Manager test cases and corresponding requirements.

Table 17: Trace Between Non-Functional Requirements Editing Manager Test Cases and Corresponding Requirements

Reqs:	NF231	NF232
TC59	■	□
Tested by usability test (Graduate Students)	□	■

Table 18 shows the trace between Non-Functional Requirements Export Manager test cases and corresponding requirements.

Table 18: Trace Between Non-Functional Requirements Export Manager Test Cases and Corresponding Requirements

Reqs:	NF241	NF242	NF243
TC63	■	■	□
TC64	■	■	□
TC65	■	■	□
TC66	■	■	□
TC67	■	■	□
Not Explicitly Tested	□	□	■

10 Trace to Modules

The following tables show the traceability between the test cases in this report and the modules defined in the Module Guide. A filled cell (■) indicates that the test case exercises or verifies behaviour involving that module; an empty cell (□) indicates it does not. The module identifiers correspond to the modules in the MG.

Table 19 shows the trace between Import Manager test cases and modules.

Table 19: Trace Between Import Manager Test Cases and Modules

TC	M1	M2	M3	M4	M5
TC1	■	□	□	■	□
TC2	■	□	□	■	□
TC3	■	□	□	■	□
TC4	■	□	□	■	□
TC5	■	□	□	■	□
TC6	■	■	□	□	■
TC7	■	□	□	□	■
TC8	□	□	■	□	□
TC9	□	□	■	□	□
TC10	□	□	■	□	□
TC11	■	■	□	■	□
TC12	■	■	□	■	□

Table 20 shows the trace between Visualization Manager test cases and modules.

Table 20: Trace Between Visualization Manager Test Cases and Modules

TC	M3	M5	M6	M9	M10	M14	M15
TC13	☐	☐	■	☐	☐	☐	☐
TC14	☐	☐	■	☐	☐	☐	☐
TC15	■	■	■	☐	☐	■	☐
TC16	■	■	■	☐	☐	■	☐
TC17	■	■	■	☐	☐	■	☐
TC18	☐	■	■	☐	☐	■	■
TC19	☐	■	■	☐	☐	■	■
TC20	☐	■	■	☐	☐	■	■
TC21	☐	■	■	■	■	■	■
TC22	☐	■	■	■	■	■	■
TC23	☐	■	■	■	■	■	■
TC24	☐	■	■	■	■	■	■
TC25	☐	☐	■	■	■	■	■
TC26	☐	☐	■	■	■	■	■

Table 21 shows the trace between Editing Manager test cases and modules (Part 1: voxel editing and history).

Table 21: Trace Between Editing Manager Test Cases and Modules (Part 1: voxel editing and history)

TC	M5	M7	M8	M9	M10	M13	M14
TC27	■	■	□	□	□	□	■
TC28	■	■	□	□	□	□	■
TC29	■	■	□	□	□	□	■
TC30	■	■	□	□	□	□	■
TC31	■	■	□	□	■	□	□
TC32	■	■	□	□	□	□	■
TC33	■	■	□	□	□	□	■
TC34	■	■	□	□	□	□	■
TC35	■	■	□	□	□	□	■
TC36	■	■	□	□	■	□	□
TC37	■	■	■	■	□	□	■
TC38	■	■	■	□	□	□	□
TC39	■	■	■	□	□	□	□
TC40	■	■	■	□	□	□	□
TC41	■	■	■	□	□	□	□
TC42	■	■	■	□	□	□	□
TC43	■	■	■	□	□	□	□
TC44	■	■	■	□	□	□	□
TC45	■	■	■	□	□	□	□
TC46	■	■	■	□	□	□	□

Table 22 shows the trace between Editing Manager test cases and modules (Part 2: layers, add/delete, reset, save).

Table 22: Trace Between Editing Manager Test Cases and Modules (Part 2: layers, add/delete, reset, save)

TC	M4	M5	M7	M10	M14
TC47	☐	■	■	■	■
TC48	☐	■	■	■	■
TC49	☐	■	■	■	■
TC50	☐	■	■	■	■
TC51	☐	■	■	■	■
TC52	☐	■	■	■	■
TC53	☐	■	■	☐	■
TC54	☐	■	■	☐	■
TC55	☐	■	■	☐	■
TC56	☐	■	■	☐	■
TC57	☐	■	■	☐	■
TC58	☐	■	■	☐	■
TC59	■	☐	☐	☐	☐
TC60	■	☐	☐	☐	☐
TC61	■	☐	☐	☐	☐
TC62	■	☐	☐	☐	☐

Table 23 shows the trace between Export Manager test cases and modules.

Table 23: Trace Between Export Manager Test Cases and Modules

TC	M9	M11	M12	M13	M14	M16
TC63	■	■	☐	☐	■	☐
TC64	■	■	☐	☐	■	☐
TC65	☐	☐	■	☐	■	■
TC66	☐	☐	■	☐	■	■
TC67	☐	☐	■	☐	■	■
TC68	☐	☐	■	☐	■	☐
TC69	☐	☐	■	☐	■	☐
TC70	☐	☐	■	☐	■	☐
TC71	☐	☐	■	☐	■	■
TC72	■	■	☐	☐	■	☐

11 Code Coverage Metrics

11.1 Python Code Coverage

The following section analyzes the code coverage of the Python code in the backend. The analysis is provided in the Figure 1 and is a measure of how much of the code is being tested, and helps us understand which parts of the code are not being tested.

Coverage report: 90%

Files | Functions | Classes

filter... hide covered

coverage.py v7.13.4, created at 2026-03-08 20:10 -0400

File ▲	statements	missing	excluded	coverage
app/__init__.py	0	0	0	100%
app/config.py	15	1	0	93%
app/models/__init__.py	2	0	0	100%
app/models/schemas.py	44	0	0	100%
app/routers/__init__.py	0	0	0	100%
app/routers/edit_router.py	144	11	0	92%
app/routers/export_router.py	29	0	0	100%
app/routers/project_router.py	70	27	0	61%
app/routers/stl_router.py	41	7	0	83%
app/services/__init__.py	0	0	0	100%
app/services/export_service.py	21	0	0	100%
app/services/history_management_service.py	39	4	0	90%
app/services/mesh_service.py	6	3	0	50%
app/services/model_editing_service.py	38	0	0	100%
app/services/model_structure_service.py	98	6	0	94%
app/services/model_tracking_service.py	51	0	0	100%
app/services/partition_manager.py	39	0	0	100%
app/services/project_management_service.py	36	5	0	86%
app/services/voxel_service.py	7	3	0	57%
Total	**680**	**67**	**0**	**90%**

coverage.py v7.13.4, created at 2026-03-08 20:10 -0400

Figure 1: Python Code Coverage

The backend codebase has an overall code coverage of 90% for statements, branches, functions, and lines. The code coverage overall met the VnV Plan's requirement of 50%. The `voxel_service.py` was the file that was the most difficult to test because it utilizes the `trimesh` library for voxelization. The team will furthur analyze the code coverage of the `voxel_service.py` file to understand which parts of the code are not being tested and improve the coverage of the file.

11.2 TypeScript Code Coverage

The following section analyzes the code coverage of the TypeScript code in the frontend. The analysis is provided in the Figure 2 and is a measure of how much of the code is being tested, and helps us understand which parts of the code are not being tested.

```
File                    | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----------------------|---------|----------|---------|---------|-------------------
All files               |   71.46 |    54.89 |   71.23 |   72.54 |
 src                    |   82.46 |       50 |   77.77 |   83.27 |
  App.tsx               |   82.46 |       50 |   77.77 |   83.27 | 113,119-124,132,157-163,169-170
 src/components         |   48.54 |    50.33 |      60 |    49.5 |
  Footer.tsx            |     100 |      100 |     100 |     100 |
  LayerEditor.tsx       |    27.1 |       22 |   29.78 |   28.09 | 108-138,146-148,155-167,175-188
  MenuBar.tsx           |     100 |    96.77 |     100 |     100 | 116,290,338
  ModelSelector.tsx     |     100 |      100 |     100 |     100 |
  NewProjectDialog.tsx  |   51.57 |    52.17 |   47.61 |   54.65 | 70-71,91-102,153-185,209-238,25
  PartitionsPanel.tsx   |     100 |    94.11 |     100 |     100 | 33
  ProjectSelector.tsx   |     100 |      100 |     100 |     100 |
  StatusMessage.tsx     |     100 |      100 |     100 |     100 |
  UploadButton.tsx      |      75 |     62.5 |     100 |   72.72 | 12,16-17
  UploadMessage.tsx     |     100 |      100 |     100 |     100 |
 src/utils              |   96.23 |    77.11 |   86.79 |   98.77 |
  api.ts                |   94.81 |    77.04 |   81.08 |   98.19 | 296-297,346
  threeUtils.ts         |   97.76 |    77.19 |     100 |   99.37 | 293
-----------------------|---------|----------|---------|---------|-------------------

=============================== Coverage summary ===============================
Statements   : 71.46% ( 879/1230 )
Branches     : 54.89% ( 376/685 )
Functions    : 71.23% ( 161/226 )
Lines        : 72.54% ( 819/1129 )
================================================================================

Test Suites: 16 passed, 16 total
Tests:       186 passed, 186 total
Snapshots:   0 total
Time:        4.326 s, estimated 7 s
Ran all test suites.
```

Figure 2: TypeScript Code Coverage

The frontend codebase has an overall code coverage of 71.46% for statements, 54.89% for branches, 71.23% for functions, and 72.54% for lines. The code coverage overall was not as high as required by the VnV Plan, however the team will focus on improving the coverage of the most critical components in the future. `LayerEditor.tsx` and `ModelViewer.tsx` were the two components that were the most difficult to test, and the team will focus on improving the coverage of these components in the future. This will be done by dividing the components into smaller, more manageable components and testing each component individually.

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

2. What pain points did you experience during this deliverable, and how did you resolve them?

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)